Deadlock

Processes that cannot proceed

Characterization, prevention, avoidance, detection and recovery

What is deadlock?

01 A set of processes becomes permanently blocked.

02 Each process waits for a resource held by another process in the set.

03 Each holder needs another resource before it can proceed and release what it holds.

04 Without intervention, none of these processes can make progress.

Ordinary waiting can end. A deadlock has a dependency cycle that cannot resolve.

A two-process example

Process	Currently holds	Waiting for
P1	Printer	Scanner
P2	Scanner	Printer

Both processes keep their device while waiting. Neither can proceed.

The resource-use model

01 Request: a process asks for a resource.

02 Use: the process performs work with the allocated resource.

03 Release: the process returns the resource for another process to use.

04 A resource type can have one or several equivalent instances.

Example: three identical printers are three instances of one resource type.

Four necessary conditions

Condition	Meaning
Mutual exclusion	At least one resource permits only one user at a time.
Hold and wait	A process keeps a resource while waiting for another.
No preemption	The system cannot forcibly take the resource away.
Circular wait	Processes form a cycle of resource dependencies.

All four are necessary for resource deadlock in this model.

The conditions in the device example

Condition	How it appears
Mutual exclusion	Only one process uses each device.
Hold and wait	P1 and P2 keep one device while requesting the other.
No preemption	Neither device can be taken away during use.
Circular wait	P1 waits for P2, and P2 waits for P1.

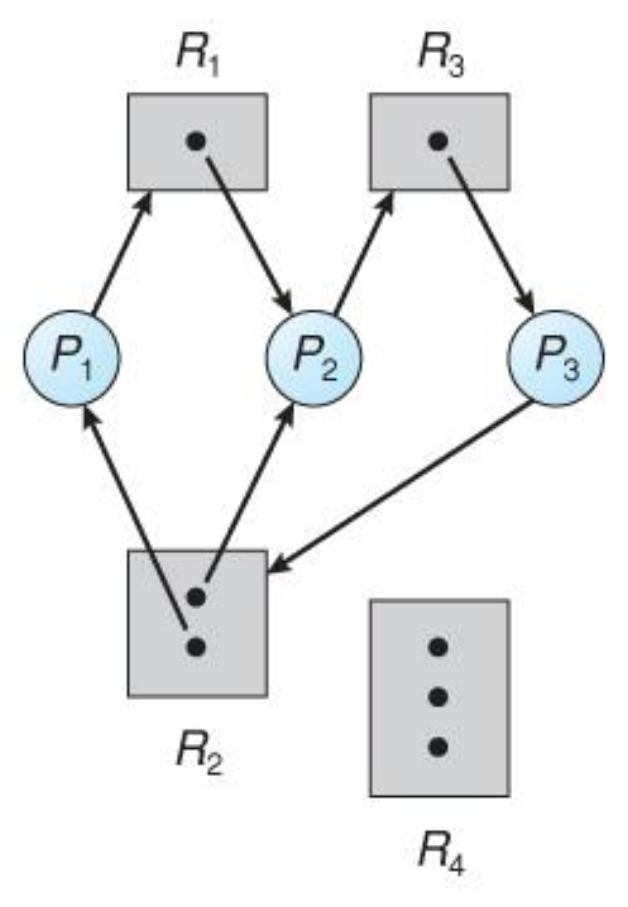
Breaking at least one necessary condition prevents this deadlock.

Resource-allocation graph notation

Graph element	Interpretation
Circle	A process
Rectangle with dots	A resource type and its instances
Process to resource edge	The process requests the resource.
Resource instance to process edge	The process holds that instance.

The direction of an edge distinguishes a request from an allocation.

A graph with deadlock



P1 waits for R1 held by P2.

P2 waits for R3 held by P3.

P3 waits for R2. Both instances belong to blocked processes.

P1, P2 and P3 cannot proceed.

Cycles and deadlock

Resource-allocation graph	Conclusion
No directed cycle	No resource deadlock
Cycle, one instance per type	Deadlock exists.
Cycle, multiple instances possible	A cycle alone does not prove deadlock.

With multiple instances, another process may finish and release a needed instance.

Ways to handle deadlock

Strategy	Main idea
Prevention	Restrict resource use to break a necessary condition.
Avoidance	Grant requests only when the resulting state is safe.
Detection	Allow allocations, then check for existing deadlocks.
Recovery	Break a detected deadlock and restore progress.

Prevention and avoidance act before deadlock. Recovery acts after detection.

Prevention methods

Condition to break	Possible policy
Mutual exclusion	Use sharing when the resource supports it.
Hold and wait	Acquire all needed resources together before starting.
No preemption	Reclaim resources when safe to save and restore their state.
Circular wait	Require a common order for requesting resources.

Exclusive devices cannot always become shareable, and preemption is not always practical.

Circular-wait prevention example

Policy	Effect
Order resources	Printer has rank 1. Scanner has rank 2.
P1 request order	Acquire printer before requesting scanner.
P2 request order	Also acquire printer before requesting scanner.
Result	One process may wait, but it does not hold the scanner while waiting for the printer.

A consistent global resource order prevents circular wait.

Costs of prevention

01 Requesting everything together can leave allocated resources idle.

02 Some processes may wait a long time for all their resources.

03 Preempting a resource can require rollback and repeated work.

04 An agreed resource order can restrict program design.

Prevention simplifies safety by limiting how processes request resources.

Deadlock avoidance

01 The system checks each request before it commits the allocation.

02 An avoidance algorithm uses information about possible future demands.

03 A safe allocation leaves a way for every process to finish.

04 An unsafe allocation must wait, even when the requested resources are free.

Banker’s Algorithm uses each process’s declared maximum demand.

Safe, unsafe and deadlocked states

State	Meaning
Safe	At least one sequence lets all processes finish within their maximum claims.
Unsafe	No such guaranteed completion sequence exists.
Deadlocked	Processes are already blocked in an unresolvable dependency.

An unsafe state may lead to deadlock. It is not necessarily deadlocked yet.

Banker’s Algorithm data

Data structure	What it records
Available	Free instances of each resource type
Allocation	Instances each process currently holds
Max	Maximum demand declared by each process
Need	Remaining possible demand: Max − Allocation

Compare vectors component by component when there are several resource types.

The safety test

01 Set Work = Available and mark every process unfinished.

02 Find an unfinished process whose Need is at most Work.

03 Pretend it finishes, then add its Allocation to Work and mark it finished.

04 Repeat. The state is safe only if every process can finish.

Use Need ≤ Work, including equality.

Worked example: initial state

Process	Allocation	Max	Need
P0	2	7	5
P1	3	5	2
P2	2	3	1

Total instances = 10. Allocated = 7. Available = 3.

Worked example: a safe sequence

Process chosen	Need	Work before	Work after
P1	2	3	6
P0	5	6	8
P2	1	8	10

One safe sequence is P1, P0, P2. Other safe sequences may exist.

Checking a new resource request

01 Reject a request that exceeds the process's remaining Need.

02 If the request exceeds Available, the process must wait.

03 Otherwise, simulate the allocation and run the safety test.

04 Commit a safe allocation. Restore the old state and make the process wait if unsafe.

Free resources alone do not guarantee a safe allocation.

An available request can be unsafe

Process	New allocation	New Need
P0	5	2
P1	3	2
P2	2	1

P0 requests 3. Available becomes 0. No process passes the safety test. Defer the request.

Deadlock detection

01 The system checks whether current allocations and waits contain a deadlock.

02 For one instance per resource type, a wait-for graph can detect cycles.

03 An edge from P1 to P2 means P1 waits for a resource held by P2.

04 A cycle in this single-instance wait-for graph identifies deadlock.

Detection checks current waiting requests. Avoidance checks future maximum needs.

Detection with multiple instances

01 Start with Work = Available and the current outstanding Request matrix.

02 Ignore processes holding no resources when identifying deadlock participants.

03 Repeatedly simulate finishing a process with Request ≤ Work and release its Allocation.

04 If no more can finish, unfinished resource-holding processes are deadlocked in this model.

Use Request for detection and Need for the Banker’s safety test.

Recovery methods

Method	Action	Cost or constraint
Terminate all	Abort all deadlocked processes.	Large loss of work
Terminate one at a time	Abort a victim, then check again.	Repeated detection
Resource preemption	Reclaim selected resources.	Needs safe state restoration

Victim selection can consider priority, completed work and recovery cost.

Rollback and starvation in recovery

01 Rollback restores a process or transaction to an earlier checkpoint.

02 A process may need to repeat work after a resource is reclaimed.

03 Repeatedly choosing the same victim can cause starvation.

04 Victim selection should account for earlier rollbacks or aborted attempts.

Preemption is suitable only when the system can restore a consistent state.

Example: database transactions

Step	Transaction T1	Transaction T2
First lock	Locks row A	Locks row B
Next request	Waits for row B	Waits for row A
Recovery	One transaction can continue.	The database aborts a victim.
Retry policy	Acquire rows in a common order.	Retry the aborted transaction.

The database can choose either transaction as the victim.

Prevention, avoidance and detection

Approach	Decision basis	When it acts
Prevention	Rules for resource acquisition	Before a cycle can form
Avoidance	Maximum needs and a safety test	Before granting a request
Detection + recovery	Current waits and allocations	After deadlock occurs

Prevent the conditions, avoid unsafe allocations, or detect and recover.